**Statement of Qualifications**

Uriah Raffer | JC-523777 | System Security Associate

**1. Describe how you would work with a customer with limited technical experience who needs assistance with technology you support.**

I'd start with what the customer is trying to accomplish, not with the technology. When someone with limited technical experience asks for help, the problem they describe and the problem that actually exists are often two different things, so I listen first and ask questions in plain terms, like what they were doing when things stopped working and what they're trying to get done. From there I translate the technical picture into what it means for them and skip the jargon. Nobody gets talked down to. Most people don't need to understand how a system works. They need to know what's wrong, what I'm doing about it, and what happens next, so I keep them informed at each step instead of leaving them wondering.

That's how I helped my neighbor Jason. He'd upgraded to a faster internet plan but wasn't getting the speeds he was paying for, and he had no idea why. Troubleshooting it, I traced the problem to his aging equipment, which was built for a slower connection than the one he'd just bought, and I explained it in terms of what it meant for him. The service was fine. His own hardware was the limit, and he was paying for speed it couldn't deliver. Once that clicked, the fix made sense to him. I upgraded the hardware and his connection finally ran at full speed.

Block Advisors put that approach under real pressure. I prepared about 70 tax returns over a season for clients who mostly were not familiar with the process and anxious about getting something wrong. My job was to take rules I understood and they didn't, and turn them into plain answers about their own situation. Keeping people steady through the process mattered as much as the paperwork. Being upfront about where things stood and what came next kept clients calmer than rushing ever did.

The same plain-language habit runs through my technical work. I keep documentation for the projects I build on GitHub, with setup guides and a running log of the problems I hit and how I fixed them, so someone who wasn't there when I built it can follow it. Writing for a reader who doesn't share your context is the same skill as helping a customer who doesn't. That's the approach I'd bring to every CalSTRS customer.

**2. Describe your experience with account provisioning and system access administration.**

My experience with account provisioning and access administration comes from environments I built and ran myself, plus my security internship. I created and maintained users and groups in a Windows domain I stood up from scratch, and I administered access, rights, and privileges in an Amazon Web Services (AWS) environment through Identity and Access Management (IAM).

The domain was an Active Directory (AD) environment in my homelab, built on Windows Server 2022. I installed the operating system and promoted two domain controllers myself, then joined the Windows 11 clients to the domain, so I've handled the account side of bringing a machine and its user into a managed environment from

nothing. I built separate organizational units (OUs) for mock Finance and Sales departments and created a user account in each, then managed the groups and memberships those accounts belonged to. I handled the password side of account administration too, performing resets and enforcing a policy that required at least 16 characters with uppercase, lowercase, numbers, and symbols. Remote access to my machines still runs through Tailscale and RustDesk, which I set up and manage myself, so reaching any of my systems means coming in as an authorized device on my private network instead of through an exposed port.

In my security internship, which runs on enterprise Microsoft Azure and Tenable infrastructure, I worked at the command line in PowerShell to bring a Windows 11 system into compliance with the Defense Information Systems Agency Security Technical Implementation Guide (DISA STIG) baseline, remediating 10 failing controls and verifying each fix with a rescan. Two of those controls were account and access settings, a minimum password length and a restriction on remote Security Account Manager (SAM) calls.

On AWS I administered access the way an enterprise would. I kept the root account out of daily work, using an IAM user for hands-on tasks and reserving root for the few actions that actually required it. My best provisioning decision came from catching my own overprovisioning. My Terraform automation was running under a broad administrator identity with long-lived credentials, so one leaked credential would have exposed the whole account. I created a dedicated role scoped to only the permissions the automation needed, backed by single sign-on (SSO) issuing temporary credentials, so the automation no longer held a standing credential at all. I also removed standing Secure Shell (SSH) access from my servers and replaced it with AWS Systems Manager Session Manager, where IAM decided who could open a session and every session was auditable in CloudTrail.

In a self-hosted GLPI lab, an IT Service Management (ITSM) ticketing platform I installed and configured, I rotated all four default accounts off their publicly documented passwords. The training behind this work is a B.S. in Network Engineering and Security along with CompTIA Security+, Network+, A+, and Cloud+ certifications. Underneath all of it is one rule. Access gets granted deliberately and scoped to what an identity actually needs, so only authorized personnel reach the systems that matter. That's the standard I'd hold at CalSTRS.